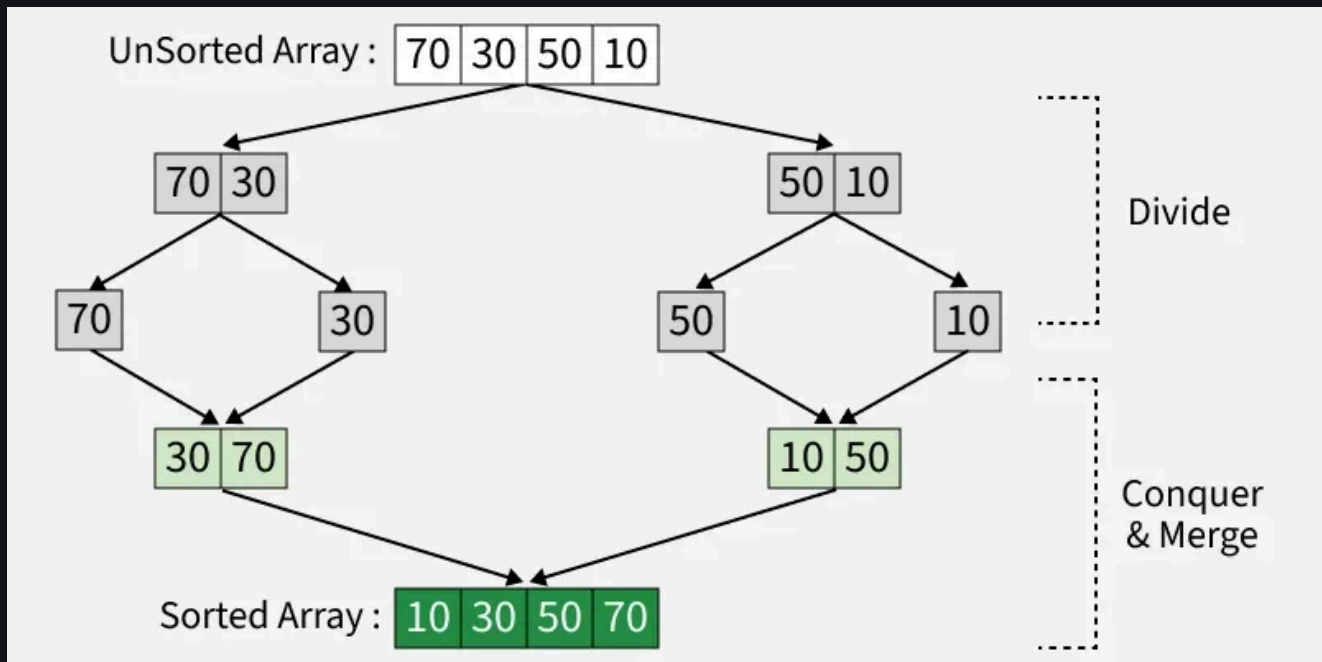


Merge Sort

Last Updated : 3 Oct, 2025



Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the Divide and Conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.



Here's a step-by-step explanation of how merge sort works:

1. **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
2. **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
3. **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Let's sort the array or list [38, 27, 43, 10] using Merge Sort



Let's look at the working of above example:

Divide:

- *[38, 27, 43, 10] is divided into [38, 27] and [43, 10] .*
- *[38, 27] is divided into [38] and [27] .*
- *[43, 10] is divided into [43] and [10] .*

Conquer:

- *[38] is already sorted.*
- *[27] is already sorted.*
- *[43] is already sorted.*
- *[10] is already sorted.*

Merge:

- *Merge [38] and [27] to get [27, 38] .*
- *Merge [43] and [10] to get [10,43] .*
- *Merge [27, 38] and [10,43] to get the final sorted list [10, 27, 38, 43]*

Therefore, the sorted list is *[10, 27, 38, 43]* .

C++ C Java Python C# JavaScript

```
#include <iostream>
#include <vector>
using namespace std;

// Merges two subarrays of arr[].
// First subarray is arr[left..mid]
// Second subarray is arr[mid+1..right]
void merge(vector<int>& arr, int left,
           int mid, int right){

    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temp vectors
    vector<int> L(n1), R(n2);

    // Copy data to temp vectors L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0;
    int k = left;
```

```

// Merge the temp vectors back
// into arr[left..right]
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    }
    else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

// Copy the remaining elements of L[],

```

[DSA](#)
[Practice Problems](#)
[C](#)
[C++](#)
[Java](#)
[Python](#)
[JavaScript](#)
[Data Science](#)
[Machine Learning](#)
[Courses](#)



Sign In

```

        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[],
    // if there are any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

// begin is for left index and end is right index
// of the sub-array of arr to be sorted
void mergeSort(vector<int>& arr, int left, int right){

    if (left >= right)
        return;

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// Driver code
int main(){

    vector<int> arr = {38, 27, 43, 10};
    int n = arr.size();

    mergeSort(arr, 0, n - 1);
    for (int i = 0; i < arr.size(); i++)
        cout << arr[i] << " ";
    cout << endl;
}

```

```
    return 0;
}
```

Output

10 27 38 43

Recurrence Relation of Merge Sort

The recurrence relation of merge sort is:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1 \end{cases}$$

- $T(n)$ Represents the total time taken by the algorithm to sort an array of size n .
- $2T(n/2)$ represents time taken by the algorithm to recursively sort the two halves of the array. Since each half has $n/2$ elements, we have two recursive calls with input size as $(n/2)$.
- $O(n)$ represents the time taken to merge the two sorted halves

Complexity Analysis of Merge Sort

Time Complexity:

- **Best Case:** $O(n \log n)$, When the array is already sorted or nearly sorted.
- **Average Case:** $O(n \log n)$, When the array is randomly ordered.
- **Worst Case:** $O(n \log n)$, When the array is sorted in reverse order.

Auxiliary Space: $O(n)$, Additional space is required for the temporary array used during merging.

Applications of Merge Sort:

- Sorting large datasets
- [External sorting](#) (when the dataset is too large to fit in memory)
- Used to solve problems like [Inversion counting](#), [Count Smaller on Right](#) & [Surpasser Count](#)
- Merge Sort and its variations are used in library methods of programming languages. Its variation [TimSort](#) is used in Python, Java Android and Swift. The main reason why it is preferred to sort non-primitive types is stability which is not there in QuickSort. [Arrays.sort in Java](#) uses QuickSort while [Collections.sort](#) uses MergeSort.
- It is a preferred algorithm for sorting Linked lists.
- It can be easily parallelized as we can independently sort subarrays and then merge.
- The merge function of merge sort to efficiently solve the problems like [union and intersection of two sorted arrays](#).

Advantages and Disadvantages of Merge Sort

Advantages

- **Stability** : Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
- **Guaranteed worst-case performance**: Merge sort has a worst-case time complexity of $O(N \log N)$, which means it performs well even on large datasets.
- **Simple to implement**: The divide-and-conquer approach is straightforward.
- **Naturally Parallel** : We independently merge subarrays that makes it suitable for parallel processing.

Disadvantages

- **Space complexity**: Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
- **Not in-place**: Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- Merge Sort is **Slower than QuickSort in general as** QuickSort is more cache friendly because it works in-place.

Quick Links:

- [Merge Sort Based Coding Questions](#)
- [Bottom up \(or Iterative\) Merge Sort](#)
- [Sorting Interview Questions](#)
- [Quiz on Merge Sort](#)

Suggested Quiz

🔄 4 Questions

What is the average time complexity of Merge Sort

- ☐ A $O(n)$
- ☐ B $O(n^2)$
- ☐ C $O(n \log n)$
- ☐ D $O(2^n)$

1/4

< Previous Next >

💬 Comment

 **kartik**

👍 1.34k



Explore

DSA Fundamentals

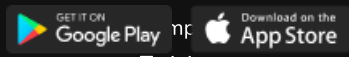


Data Structures	▼
Algorithms	▼
Advanced	▼
Interview Preparation	▼
Courses	▼



Corporate & Communications Address:
A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

About Us
Legal
Privacy
Policy
Contact Us
Advertise with us
GfG
Corporate Solution
Training Program

Explore

POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials

Programming Languages
DSA
Web
Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses

ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner

Interview Corner
Aptitude
Puzzles
GfG 160
System Design